

DeepProtein Paper Summary for Protein Sequence Augmentation Research

Paper: DeepProtein: Deep Learning Library and Benchmark for Protein Sequence Learning

Lens: Should this benchmark/library help us study augmentation type and intensity?

1. Main contribution of this paper

This is primarily a benchmark and library/tooling paper, not a paper about a new augmentation method. The authors introduce DeepProtein, a user-friendly deep learning library and benchmark for protein sequence learning. Its goal is to make many protein prediction tasks easier to run with a shared interface, standardized splits, multiple model families, and reproducible experiment scripts.

- Core contribution: a library that wraps protein datasets, preprocessing, featurization, model training, evaluation, and inference into a simpler interface.
- Benchmark contribution: a broad comparison across sequence models, graph models, pretrained protein language models, general chemistry LLMs, and their own fine-tuned DeepProt-T5 models.
- Model contribution: DeepProt-T5, a family of Prot-T5-XL models fine-tuned on individual benchmark tasks. This is useful as a strong reference point, but it is not the main thing we need for an augmentation project.
- Research relevance: DeepProtein is useful mainly as infrastructure and as a menu of candidate datasets/tasks. It does not directly answer how augmentation type or augmentation intensity changes performance.

Gap claimed vs prior benchmarks: The authors argue that DeepPurpose is useful for drug discovery but has limited protein-only task coverage; FLIP and TAPE cover narrower benchmark scopes; PEER covers many protein sequence tasks but is less user-friendly and focuses more on CNN/Transformer/ESM-style sequence methods. DeepProtein claims to broaden the task set, include more model families, add TDC datasets, and provide a simpler interface.

Why this matters for us: For our project, the value of this paper is not that we should reproduce its full benchmark. The value is that it identifies several standardized datasets and simple baselines where augmentation experiments could be inserted cleanly.

2. Datasets used in DeepProtein

The paper organizes datasets into protein function prediction, localization, protein-protein interaction, epitope/paratope residue-level prediction, antibody developability, CRISPR repair outcome, and protein structure prediction. The paper states that DeepProtein follows PEER and TDC train/validation/test splits. Many datasets are sequence-only at the model input level, although some tasks originate from structure-related resources such as PDB or SAbDab.

Dataset	Task group	Task type	Size / split	Input style	Usefulness for our project
Fluorescence	Protein function	Regression	54,025 sequences;	Raw protein	Good candidate:

DeepProtein summary for augmentation research

			split 21,446 / 5,362 / 27,217	sequence; label is log fluorescence intensity	moderate size, well-known, continuous output; intensity sweeps could be meaningful.
Stability	Protein function	Regression	68,934 sequences; split 53,571 / 2,512 / 12,851	Raw protein sequence; label is protein stability	Good but larger; biologically relevant for mutation robustness.
Beta-lactamase	Protein function	Regression	5,198 sequences; split 4,158 / 520 / 520	Raw protein sequence; fitness/activity score for mutants	Very attractive for augmentation because it is directly mutation-related and small enough for fast experiments.
Solubility	Protein function	Binary classification	71,419 sequences; split 62,478 / 6,942 / 1,999	Raw protein sequence; soluble vs not soluble	Best simple classification candidate if the project must be classification-focused.
Subcellular	Localization	10-class classification	13,961 samples; split 8,945 / 2,248 / 2,768	Raw protein sequence; cellular location label	Strong candidate: manageable size, classification, biologically meaningful labels.
Binary localization	Localization	Binary classification	8,634 samples; split 5,161 / 1,727 / 1,746	Raw protein sequence; membrane-bound vs soluble	Easiest classification dataset; may be too simple but useful for pilot experiments.
PPI Affinity	Protein-protein interaction	Regression on protein pairs	2,682 pairs; split 2,127 / 212 / 343	Two protein sequences per sample; real-valued affinity/activity	Small but pair-input pipeline adds complexity.
Yeast PPI	Protein-protein interaction	Binary classification on protein pairs	2,172 pairs; split 1,668 / 131 / 373	Two protein sequences per sample	Small, but augmentation is more complicated because each sample has two proteins.
Human PPI	Protein-protein interaction	Binary classification on protein pairs	7,348 pairs; split 6,844 / 277 / 227	Two protein sequences per sample	More realistic but still pair-input complexity.
IEDB	Epitope prediction	Residue-level binary classification	3,159 antigens; split 2,211 / 316 / 632	Protein sequence with per-residue labels	Not ideal for first project; augmentation must preserve residue-label alignment.
PDB-Jespersen	Epitope prediction	Residue-level binary classification	447 antigens; split 313 / 45 / 89	PDB-derived antigen sequences with	Small and label-alignment sensitive; avoid

DeepProtein summary for augmentation research

				per-residue labels	initially.
SAbDab-Liberis	Paratope prediction	Residue-level binary classification	1,023 antibody chain samples; split 716 / 102 / 205	Antibody heavy/light chain sequence data with per-residue labels	Avoid initially; antibody-specific and label-alignment sensitive.
TAP	Antibody developability	Regression	242 antibodies; split 169 / 24 / 48	Heavy and light chain sequences; real-valued developability	Too small for broad augmentation claims.
SAbDab-Chen	Antibody developability	Regression/binary developability setting in paper text/tables	2,409 antibodies; split 1,686 / 241 / 482	Heavy/light chain sequences from SAbDab	Domain-specific; useful later, not ideal first dataset.
CRISPR-Leenay	CRISPR repair outcome	Regression	1,521 DNA sequences; split 1,065 / 152 / 304	DNA sequence, not protein sequence	Not aligned with protein sequence augmentation; avoid.
Fold	Protein structure prediction	1,195-class classification	13,766 samples; split 12,312 / 736 / 718	Protein sequence mapped to fold class	Interesting but hard: many classes, low baseline accuracy for simple models.
Secondary structure	Protein structure prediction	Residue-level 3-class classification	11,361 samples; split 8,678 / 2,170 / 513	Per-residue local structure labels	Avoid initially because augmentation must preserve per-residue labels.

Most relevant datasets: For a clean augmentation project, the best candidates are Beta-lactamase, Solubility, Subcellular, Binary localization, and possibly Stability. These are sequence-input tasks with clear labels and do not require pair handling or residue-level label alignment.

3. Benchmark setup

- Splits: the paper says it follows PEER benchmark splits and TDC splits. This is helpful because it avoids us inventing arbitrary splits.
- Runs: most reported results use multiple random seeds, often with means and standard deviations. The paper also describes statistical tests comparing the best and second-best methods.
- Training: default maximum training epoch is 100; Adam optimizer; learning rate 1e-4 for sequence models and 1e-5 for structure-based models; batch size 32 for most models. DeepProt-T5 fine-tuning uses learning rate 2e-5 and batch size 10.
- Hardware: experiments used a 40GB NVIDIA A40 and 24GB RTX 3090s. This matters because some models are not feasible on smaller hardware.
- Metrics: regression tasks use Spearman correlation, R-squared, and/or MAE. Classification tasks use accuracy, PR-AUC, ROC-AUC, and averaged/macro F1 depending on task type.

Can we use this directly?: Probably yes for dataset access and baseline training, assuming the GitHub library runs cleanly. But for a summer project, we should not use the entire benchmark. Use DeepProtein as a dataset/model loader, then run a focused augmentation study on one or two tasks.

4. Models compared

Model category	What it is	Performance takeaway	Practical for us?
CNN	1D convolution over amino acid sequence tokens; captures local motifs/patterns.	Often strong among simple models; performed well on several function and regression tasks.	Yes. Best lightweight baseline for a summer project.
CNN-RNN / GRU	CNN layers plus bidirectional GRU to capture local and longer-range sequence patterns.	Strong on some tasks, especially TAP and secondary structure; sometimes better than plain CNN.	Maybe. Useful second baseline if time allows.
Transformer	Small transformer encoder over tokenized sequence.	Mixed. Not always better than CNN/CNN-RNN. Sometimes weak in their benchmark.	Maybe, but not necessary. Could distract from augmentation question.
GNNs: GCN, GAT, NeuralFP, AttentiveFP, MPNN	They convert/featurize protein strings into graph-like inputs and apply graph neural networks.	Generally inferior to sequence/protein language models in this benchmark. Some OOM issues.	No for first project. Adds complexity without clear payoff.
Graph transformers: PAGTN, Graphormer	Graph attention/transformer architectures for graph-structured representations.	Often inefficient or OOM; not central for sequence augmentation.	Avoid.
Protein language models: ESM-1b, ESM-2-650M, Prot-T5-XL, ProtBert	Pretrained models provide embeddings for downstream prediction; paper uses fixed upstream embeddings for these comparisons.	Usually strong, especially ESM-2 and Prot-T5. They are important to compare against simple baselines.	Yes, but preferably frozen embeddings rather than full fine-tuning.
DeepProt-T5	Their fine-tuned Prot-T5-XL family with dynamic upstream embeddings.	Very strong on several tasks, especially Fold and Beta-lactamase.	Probably not for our baseline; too strong/expensive and may obscure augmentation effects. Use as context, not core method.
ChemLLM-7B and LlaSMol-Mistral-7B	Prompted decoder LLMs for protein property prediction without task-specific fine-tuning.	Poor or unstable in many tasks. Included mainly as exploratory comparison.	Avoid. Not relevant to controlled augmentation intensity sweeps.

Key modeling takeaway: A clean setup would use one lightweight model, likely CNN, and one pretrained embedding model, likely frozen ESM-2 or Prot-T5 embeddings with a simple classifier. This directly tests whether augmentation helps small models more than pretrained representations.

5. Most realistic datasets for augmentation experiments

Decision category	Recommended dataset(s)	Reason
Easiest for fast experimentation	Binary localization or Beta-lactamase	Binary localization is small and classification-based; Beta-lactamase is small and mutation-centered.

Best simple classification task	Solubility	Large enough for training, simple label format, easy metric reporting. But it may be computationally larger.
Best biological fit for augmentation	Beta-lactamase or Stability	Both relate naturally to sequence mutations and fitness/stability changes. Augmentation realism matters here.
Best publication-quality direction	Beta-lactamase plus one classification task such as Subcellular or Solubility	This lets us discuss mutation realism on a mutation-sensitive task and classification behavior on a broader protein property task.
Probably too complex initially	PPI, epitope/paratope, secondary structure, CRISPR, TAP	Pair inputs, residue-level labels, DNA rather than protein, or very small sample sizes complicate the experiment.

6. How augmentation would fit into DeepProtein

DeepProtein itself is not an augmentation paper. Augmentation would need to be inserted into the data loading/training pipeline, ideally as a transform applied to training sequences only. The validation and test sets must remain unaugmented.

- Best insertion point: online augmentation during training, applied after loading the raw protein sequence but before tokenization/embedding. This lets us sample a fresh mutation each epoch and control intensity.
- Alternative insertion point: offline augmentation, where each training sequence is expanded into several precomputed variants. This is easier to audit but increases dataset size and storage.
- For frozen ESM/Prot-T5 embeddings: either augment sequences first and recompute embeddings, or do no augmentation at the embedding level. Recomputing embeddings is more expensive but scientifically cleaner.
- For CNN/CNN-RNN: augmentation is easy because the model consumes tokenized sequences directly.
- Do not augment validation/test sequences. The point is to test whether augmented training improves generalization to real held-out proteins.

Would DeepProtein make this easier than PEER?: Possibly. DeepProtein appears designed around a simpler interface and includes datasets from PEER and TDC. If the library works, it may be easier than using PEER directly. But we should still keep the project narrow by choosing one or two datasets.

Augmentations to test: Start with random substitution, conservative substitution, deletion, insertion, local shuffle, crop/subsequence, and possibly back-translation style synonymous nucleotide perturbation only if implementation time allows. The key novelty should be intensity sweeps, not inventing a new augmentation algorithm.

7. Weaknesses of using DeepProtein for our project

- Too broad: the benchmark spans many tasks, models, and metrics. That is useful for library validation but risky for a focused summer research question.

- Not augmentation-centered: the paper gives infrastructure but does not directly study augmentation effects, destructive thresholds, or biological realism of perturbations.
- Compute mismatch: some reported experiments use large GPUs and expensive models. Full DeepProt-T5 fine-tuning is not necessary for us.
- GNN setup may be questionable: the authors themselves note that graph representations from sequence/string inputs may be ill-defined without true 3D coordinate information, and GNNs often perform poorly.
- Potential reproducibility risk: library papers can be very useful but may have environment/version friction. We should test installation early before committing.
- Metric fragmentation: using too many tasks would force us to juggle regression, binary classification, multiclass classification, residue-level classification, and pair prediction metrics. That would dilute the story.
- Some tasks are not protein sequence classification in the sense we care about, such as CRISPR DNA repair outcome prediction.

8. Should we build on DeepProtein or use something simpler?

Option	Pros	Cons	Recommendation
DeepProtein	Good if installation works. Gives many datasets and baselines in one place.	Can become too broad; not augmentation-native.	Best infrastructure candidate, but use only a small slice.
PEER	Standard benchmark used by the augmentation paper and DeepProtein.	Less user-friendly; may be more engineering-heavy.	Good scientifically, but DeepProtein may be easier if it wraps PEER tasks cleanly.
TAPE	Classic and well-known protein benchmark.	Older and narrower; may not feel as current.	Useful backup, especially for remote homology or fluorescence/stability-style tasks.
FLIP	Good for fitness landscape and mutation-effect tasks.	Narrower and more specialized.	Strong option if the project pivots toward mutation robustness/fitness landscapes.
Custom Pfam	Clear protein family classification, large and intuitive.	More preprocessing/splitting decisions; may need more data engineering.	Good if you want a pure classification story, but less plug-and-play.

Final recommendation: Try DeepProtein first as the practical path. If installation and dataset loading are smooth, use it for one dataset and one or two baselines. If it becomes messy, pivot to a simpler custom Pfam or FLIP/TAPE setup. Do not reproduce the full DeepProtein benchmark.

9. Clean summer research experiment using this paper

A realistic project should isolate augmentation as the main independent variable. The final paper should not look like a smaller DeepProtein benchmark. It should look like a controlled study of augmentation intensity and biological plausibility.

Design choice	Recommendation
Dataset	Primary: Beta-lactamase if regression is acceptable; otherwise Solubility or Subcellular for classification. Pilot:

DeepProtein summary for augmentation research

	Binary localization for quick debugging.
Baseline model	1D CNN from DeepProtein or a simple CNN classifier implemented independently.
Pretrained comparison	Frozen ESM-2 or Prot-T5 embeddings plus a small MLP/logistic classifier. Avoid full fine-tuning initially.
Augmentations	Random substitution, conservative substitution using amino acid groups, deletion, insertion, local shuffle, crop/subsequence. Optional: back-translation if time allows.
Intensity sweep	Use fixed perturbation rates such as 0%, 1%, 2%, 5%, 10%, 15%, 20%. For each augmentation type, plot performance vs intensity.
Evaluation metric	Classification: macro F1 and PR-AUC/accuracy depending on dataset. Regression: Spearman correlation and MAE. Always report mean/std over seeds.
Main hypothesis	Low-intensity biologically plausible augmentation may improve generalization, especially for lightweight models, but high-intensity or biologically unrealistic transformations will eventually become destructive. Pretrained embeddings may be less sensitive or benefit less.
Expected contribution	A controlled augmentation intensity benchmark that identifies where augmentation switches from regularization to label-destroying noise. This is not directly answered by DeepProtein or the APA paper.

Bottom line

DeepProtein is useful for your project because it may provide a practical route to datasets and baseline models. It does not solve your research question. The paper is broad, model-centric, and benchmark/library oriented. Your opportunity is to use one small part of this infrastructure to run a cleaner scientific experiment: controlled augmentation type and intensity sweeps, with biological plausibility as the interpretive lens.

Do not let DeepProtein pull the project into benchmarking every model on every dataset. Use it as a tool, not as the research question.

Specific paper references used: Table 1 compares DeepProtein with DeepPurpose, FLIP, TAPE, PEER, and TDC. Section 3.1 lists datasets and task definitions. Figure 1 gives the overall framework. Figure 2 distinguishes fixed protein language model embeddings from dynamically fine-tuned DeepProt-T5. Tables 2-7 report model results across tasks. Table 8/Figure 4 report time and memory costs. Figure 5 shows selected training curves. Appendix B describes dataset processing and training flow; Table 10 lists default model configurations.